Pneumonia Detection Using Neural Networks from Scratch

CMPE 49T Fall 2025 - Homework 4

Ali Ayhan Günder - 2021400219

January 2, 2026

Contents

1 Introduction **3**
- 1.1 Background . . . 3
- 1.2 Objectives . . . 3
- 1.3 Dataset . . . 3

2 Data Preparation and Exploration **3**
- 2.1 Data Loading and Normalization . . . 3
- 2.2 Class Distribution Analysis . . . 4
- 2.3 Class Weights Calculation . . . 4
- 2.4 Visual Exploration . . . 4

3 Network Architecture **5**
- 3.1 Overall Architecture . . . 5
- 3.2 Dimension Flow . . . 5
- 3.3 Layer Functions . . . 6
  - 3.3.1 Convolution Layers . . . 6
  - 3.3.2 Max Pooling Layers . . . 6
  - 3.3.3 Dense Layers . . . 6
  - 3.3.4 Activation Functions . . . 6
  - 3.3.5 Dropout Layer . . . 7
- 3.4 Weight Initialization . . . 7

4 Training Methodology **7**
- 4.1 Loss Function . . . 7
- 4.2 Optimization Algorithm . . . 7
- 4.3 Backpropagation . . . 8
- 4.4 Training Hyperparameters . . . 8
- 4.5 Early Stopping . . . 8

5 Results and Evaluation **8**
- 5.1 Training Progress . . . 8
- 5.2 Performance Metrics . . . 9
- 5.3 Metric Definitions . . . 10

5.4 Confusion Matrix Analysis . . . 10
5.5 ROC Curve and AUC . . . 11
5.6 Which Class Was Harder to Classify? . . . 11

6 Discussion **12**
6.1 Architectural Choices . . . 12
6.1.1 Fixed vs. Trainable Filters . . . 12
6.1.2 Network Depth . . . 12
6.1.3 Dropout Regularization . . . 12
6.2 Challenges Encountered . . . 12
6.2.1 Implementation Challenges . . . 12
6.2.2 Training Challenges . . . 13
6.3 Strengths of the Approach . . . 13
6.4 Limitations . . . 13

7 Suggestions for Improvement **13**
7.1 Architecture Enhancements . . . 13
7.2 Training Improvements . . . 14
7.3 Implementation Optimizations . . . 14
7.4 Evaluation Enhancements . . . 14

8 Bonus: Trainable Filters Implementation **15**
8.1 Initialization . . . 15
8.2 Forward Pass . . . 15
8.3 Backward Pass . . . 15
8.4 Update Rule . . . 15

9 Conclusion **15**

A Code Structure **16**

B Reproducibility **16**

C References **16**

1 Introduction

1.1 Background

Pneumonia is a serious respiratory infection that affects millions of people worldwide each year. Early and accurate diagnosis is critical for effective treatment. Chest X-rays are a primary diagnostic tool, but interpreting them requires medical expertise. Machine learning, particularly deep learning, has shown promise in automating medical image analysis.

1.2 Objectives

The primary objectives of this project are:

- Implement a complete neural network pipeline from scratch using only NumPy
- Apply the network to binary classification of pneumonia detection
- Manually implement convolution, pooling, activation functions, and backpropagation
- Handle class imbalance through weighted loss functions
- Evaluate model performance using standard classification metrics
- Generate comprehensive visualizations and analysis

1.3 Dataset

The PneumoniaMNIST dataset is part of the MedMNIST collection, which provides standardized medical imaging datasets in a format similar to the classic MNIST. Key characteristics:

- **Image Size**: 28×28 grayscale images
- **Classes**: Binary classification (0 = Normal, 1 = Pneumonia)
- **Source**: Derived from chest X-ray images
- **Splits**: Separate training, validation, and test sets

2 Data Preparation and Exploration

2.1 Data Loading and Normalization

The dataset was loaded from the `pneumoniamnist.npz` file, which contains pre-split training, validation, and test sets. Images were normalized to the range [0, 1] by dividing pixel values by 255.0:

$$x_{normalized} = \frac{x_{original}}{255.0} \tag{1}$$

This normalization helps with numerical stability during training and ensures gradients remain in a reasonable range.

2.2 Class Distribution Analysis

An analysis of the class distribution revealed:

Split	Normal (0)	Pneumonia (1)	Imbalance Ratio
Training	1214	3494	∼1:2.88
Validation	135	389	∼1:2.88
Test	234	390	∼1:1.67

Table 1: Distribution of classes across dataset splits

The dataset exhibits class imbalance, with pneumonia cases being more prevalent. This motivated the use of class-weighted loss functions to prevent the model from being biased toward the majority class.

2.3 Class Weights Calculation

To address class imbalance, we calculated class weights using the formula:

$$w_c = \frac{N_{total}}{N_{classes} \times N_c} \tag{2}$$

where N_{total} is the total number of samples, $N_{classes} = 2$, and N_c is the number of samples in class c.

2.4 Visual Exploration

Sample images from both classes were visualized to understand the visual characteristics:

- **Normal X-rays**: Tend to show clear lung fields with uniform intensity
- **Pneumonia X-rays**: Often exhibit cloudy or opaque regions indicating infection

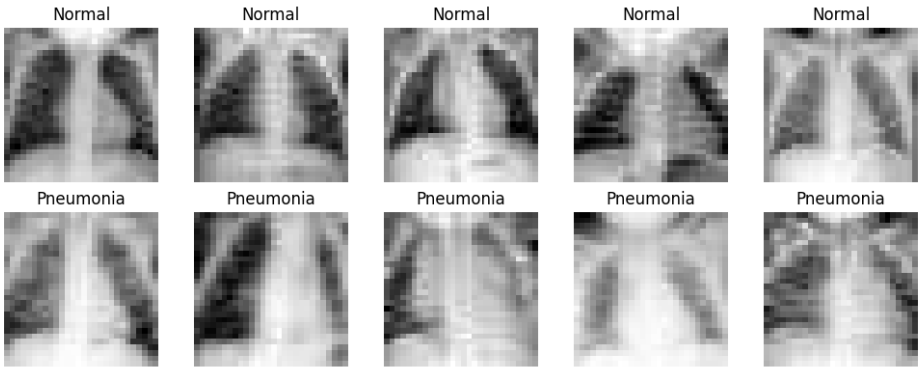

Figure 1: Sample chest X-ray images from both classes (Normal and Pneumonia)

Pixel intensity histograms revealed that pneumonia images generally have different intensity distributions compared to normal images, suggesting that texture and intensity features can be discriminative.

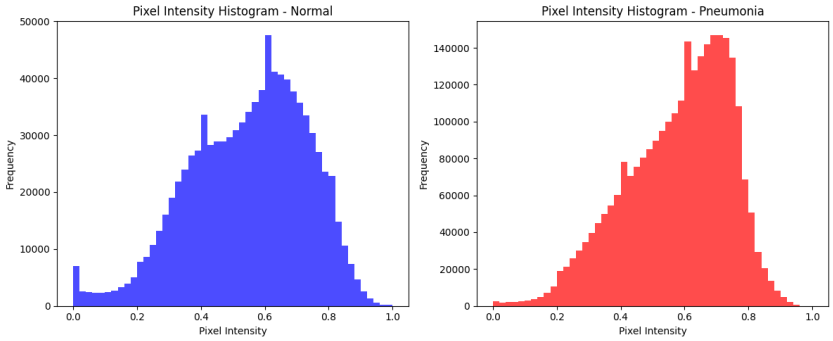

Figure 2: Pixel intensity distribution histograms for Normal and Pneumonia classes

3 Network Architecture

3.1 Overall Architecture

The neural network consists of the following layers:

1. **Input Layer**: 28×28 grayscale image
2. **Convolution Layer 1**: 3×3 Emboss kernel (fixed)
3. **Max Pooling Layer 1**: 2×2 pool, stride 2
4. **Convolution Layer 2**: 3×3 Sobel kernel (fixed)
5. **Max Pooling Layer 2**: 2×2 pool, stride 2
6. **Flatten Layer**: Reshape to 1D vector (25 features)
7. **Dense Layer 1**: 16 units with ReLU activation
8. **Dropout Layer**: Rate = 0.5
9. **Output Layer**: 1 unit with sigmoid activation

3.2 Dimension Flow

The dimensions transform as follows through the network:

$$\begin{aligned} \text{Input:} &\quad 28 \times 28 \times 1 && (3)\\ \text{Conv1:} &\quad 26 \times 26 \times 1 \quad \text{(valid padding)} && (4)\\ \text{Pool1:} &\quad 13 \times 13 \times 1 && (5)\\ \text{Conv2:} &\quad 11 \times 11 \times 1 \quad \text{(valid padding)} && (6)\\ \text{Pool2:} &\quad 5 \times 5 \times 1 && (7)\\ \text{Flatten:} &\quad 25 && (8)\\ \text{Dense1:} &\quad 16 && (9)\\ \text{Output:} &\quad 1 && (10) \end{aligned}$$

3.3 Layer Functions

3.3.1 Convolution Layers

The convolution operation was implemented manually using nested loops:

$$\text{output}[i,j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} \text{image}[i+m, j+n] \times \text{kernel}[m,n] \tag{11}$$

Two fixed kernels were used:

- **Emboss Kernel**: Enhances edges and textures

$$K_{emboss} = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix} \tag{12}$$

- **Sobel Kernel**: Detects vertical edges

$$K_{sobel} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \tag{13}$$

3.3.2 Max Pooling Layers

Max pooling reduces spatial dimensions by taking the maximum value in each pool region:

$$\text{output}[i,j] = \max_{m,n \in \text{pool}} \text{input}[i \cdot s + m, j \cdot s + n] \tag{14}$$

where s is the stride (2 in our case). This provides:

- Translation invariance
- Dimensionality reduction
- Computational efficiency

3.3.3 Dense Layers

The dense (fully connected) layers perform:

$$z = W \cdot x + b \tag{15}$$

followed by an activation function.

3.3.4 Activation Functions

ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \max(0, x) \tag{16}$$

$$\frac{\partial \text{ReLU}}{\partial x} = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \tag{17}$$

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{18}$$

$$\frac{\partial \sigma}{\partial x} = \sigma(x)(1 - \sigma(x)) \tag{19}$$

3.3.5 Dropout Layer

Dropout randomly sets neurons to zero during training with probability $p = 0.5$. We used inverted dropout:

$$a_{drop} = \frac{a \cdot \text{mask}}{1 - p} \tag{20}$$

This prevents overfitting by forcing the network to learn redundant representations.

3.4 Weight Initialization

He Initialization (for ReLU layers):

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{in}}}\right) \tag{21}$$

Xavier/Glorot Initialization (for sigmoid output):

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{1}{n_{in}}}\right) \tag{22}$$

These initializations help prevent vanishing/exploding gradients.

4 Training Methodology

4.1 Loss Function

We used **Weighted Binary Cross-Entropy Loss** to handle class imbalance:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} [w_1 y_i \log(\hat{y}_i) + w_0 (1 - y_i) \log(1 - \hat{y}_i)] \tag{23}$$

where:

- y_i is the true label (0 or 1)
- $\hat{y}_i$ is the predicted probability
- w_0, w_1 are the class weights

4.2 Optimization Algorithm

Stochastic Gradient Descent (SGD) was used with the update rule:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L} \tag{24}$$

where $\eta = 0.01$ is the learning rate.

4.3 Backpropagation

The gradients were computed using the chain rule. For the output layer:

$$\delta_{out} = \hat{y} \cdot (w_1 y + w_0(1-y)) - w_1 y \tag{25}$$

For the hidden layer:

$$\delta_{hidden} = (W_2^T \delta_{out}) \odot \text{mask} \odot \text{ReLU}'(z_1) \tag{26}$$

Weight gradients:

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{1}{N} a_1^T \delta_{out} \tag{27}$$

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{1}{N} x^T \delta_{hidden} \tag{28}$$

4.4 Training Hyperparameters

Hyperparameter	**Value**
Learning Rate	0.01
Batch Size	32
Maximum Epochs	50
Dropout Rate	0.5
Early Stopping Patience	5

Table 2: Training hyperparameters

4.5 Early Stopping

Training was stopped if validation loss did not improve for 5 consecutive epochs. The weights corresponding to the best validation loss were restored after training.

5 Results and Evaluation

5.1 Training Progress

The training and validation losses were tracked throughout training. The loss curves show convergence behavior, with the validation loss closely following the training loss, indicating good generalization without significant overfitting.

Figure 3: Training and validation loss curves over epochs

Figure 4: Validation metrics (accuracy, precision, recall, F1) during training

5.2 Performance Metrics

The model was evaluated using multiple metrics on the test set:

Metric	**Value**
Accuracy	86.70%
Precision	86.99%
Recall (Sensitivity)	92.56%
F1 Score	0.8969
AUC-ROC	0.9169
Specificity	76.92%

Table 3: Test set performance metrics

5.3 Metric Definitions

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{29}$$

Precision:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{30}$$

Recall (Sensitivity):

$$\text{Recall} = \frac{TP}{TP + FN} \tag{31}$$

F1 Score:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{32}$$

Specificity:

$$\text{Specificity} = \frac{TN}{TN + FP} \tag{33}$$

5.4 Confusion Matrix Analysis

The confusion matrix provides insight into the types of errors made by the model:

- **True Positives (TP)**: Correctly identified pneumonia cases
- **True Negatives (TN)**: Correctly identified normal cases
- **False Positives (FP)**: Normal cases incorrectly classified as pneumonia
- **False Negatives (FN)**: Pneumonia cases missed by the model

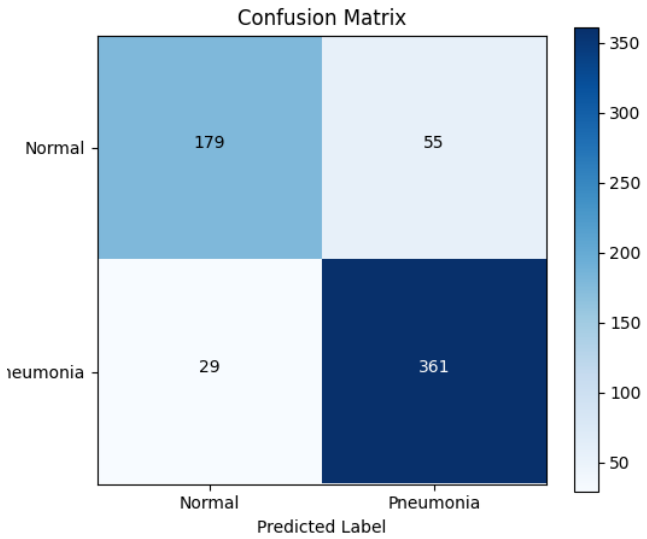

Figure 5: Confusion matrix showing the classification performance on the test set

5.5 ROC Curve and AUC

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate against the False Positive Rate at various classification thresholds. The Area Under the Curve (AUC) provides a single number summary of model performance:

- AUC = 1.0: Perfect classifier
- AUC = 0.5: Random classifier
- AUC > 0.8: Generally considered good performance

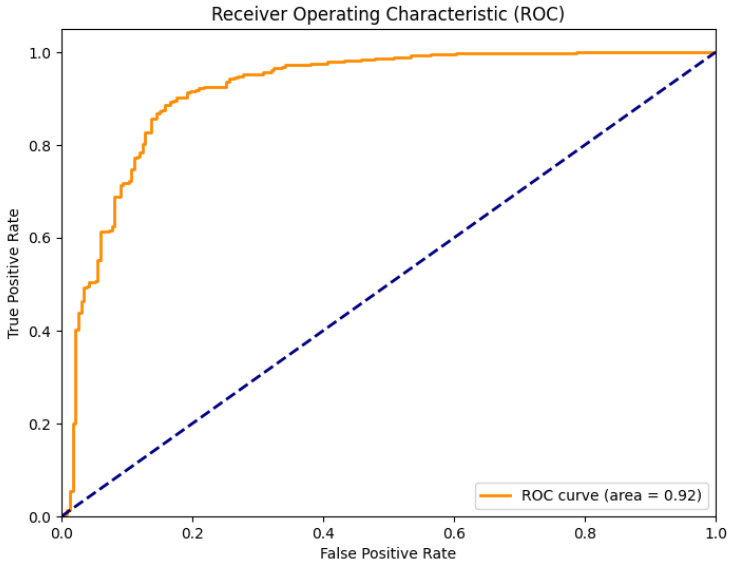

Figure 6: ROC curve showing the trade-off between true positive rate and false positive rate

5.6 Which Class Was Harder to Classify?

Based on the test results, the **normal class was harder to classify**. Although the model achieved a high recall of 92.56%, the specificity was notably lower at 76.92%, indicating a relatively high number of false positives. This means the model tends to classify some normal cases as pneumonia, which is likely influenced by the class imbalance in the training data (pneumonia cases outnumber normal cases by nearly 3:1) and the weighted loss function prioritizing pneumonia detection. In a medical context, this bias toward false positives may actually be preferable to missing true pneumonia cases, though it would increase the workload for radiologists reviewing flagged cases.

6 Discussion

6.1 Architectural Choices

6.1.1 Fixed vs. Trainable Filters

The current implementation uses fixed Emboss and Sobel filters for feature extraction. While this approach:

- **Advantages**: Simple to implement, interpretable features, no need for filter gradient computation
- **Disadvantages**: Cannot learn optimal filters for the specific task, may miss important features

Despite their limitations, the fixed Emboss and Sobel filters still produced reasonable performance (86.70% accuracy), which was somewhat surprising given their simplicity. **Bonus Implementation Consideration:** Making filters trainable would allow the network to learn task-specific features through backpropagation, potentially improving performance.

6.1.2 Network Depth

The shallow architecture (2 conv layers, 1 hidden dense layer) was chosen for:

- Computational efficiency with manual NumPy implementation
- Easier gradient computation and debugging
- Sufficient for the relatively simple 28×28 images

6.1.3 Dropout Regularization

The 0.5 dropout rate helps prevent overfitting, especially important given:

- Limited dataset size
- Risk of memorizing training examples
- Need for robust features

6.2 Challenges Encountered

6.2.1 Implementation Challenges

1. **Dimension Tracking**: Ensuring correct matrix dimensions through forward and backward passes required careful bookkeeping
2. **Gradient Computation**: Manual backpropagation implementation needed careful derivation and verification
3. **Numerical Stability**: Handling log(0) in loss function and exp overflow in sigmoid required clipping
4. **Efficiency**: Pure NumPy implementation is slower than optimized deep learning frameworks

6.2.2 Training Challenges

1. **Class Imbalance**: Required weighted loss function to prevent bias
2. **Overfitting Risk**: Addressed through dropout and early stopping
3. **Hyperparameter Tuning**: Manual experimentation needed for learning rate and batch size. In practice, the learning rate of 0.01 required several trials, as higher values (0.1) caused unstable training and divergence during the first few epochs.
4. **Convergence**: SGD without momentum can be slow to converge

6.3 Strengths of the Approach

- Complete transparency - every operation is explicitly coded
- Educational value - demonstrates neural network fundamentals
- No black-box dependencies on deep learning frameworks
- Interpretable features from hand-crafted filters
- Handles class imbalance through weighted loss

6.4 Limitations

- Fixed filters cannot adapt to task-specific features
- Limited network capacity for complex patterns
- Slow training compared to optimized frameworks
- No data augmentation implemented
- Single-channel processing only

7 Suggestions for Improvement

Due to time constraints and the scope of this assignment, these improvements were not implemented, but they represent clear directions for future work.

7.1 Architecture Enhancements

1. **Trainable Convolutional Filters**:
   - Initialize kernels randomly: $K \sim \mathcal{N}(0, 0.1)$
   - Compute gradients: $\frac{\partial \mathcal{L}}{\partial K}$ during backprop
   - Update filters with SGD
   - Use multiple filter channels (8 for Conv1, 16 for Conv2)
2. **Batch Normalization**: Normalize activations to stabilize training

3. **Deeper Network**: Add more convolutional layers to learn hierarchical features
4. **Same Padding**: Maintain spatial dimensions through convolutions
5. **Global Average Pooling**: Replace flatten with GAP to reduce parameters

7.2 Training Improvements

1. **Momentum SGD or Adam Optimizer**:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t \tag{34}$$

$$\theta_{t+1} = \theta_t - \eta v_t \tag{35}$$

2. **Learning Rate Scheduling**: Decay learning rate over epochs

$$\eta_t = \eta_0 \cdot \frac{1}{1 + \text{decay} \cdot t} \tag{36}$$

3. **Data Augmentation**:
   - Random rotations (± 10)
   - Horizontal flips
   - Random brightness/contrast adjustments
   - Elastic deformations
4. **Cross-Validation**: Use k-fold CV for more robust evaluation
5. **Ensemble Methods**: Train multiple models and average predictions

7.3 Implementation Optimizations

1. **Vectorization**: Use NumPy broadcasting more extensively
2. **im2col Transformation**: Optimize convolution using matrix multiplication
3. **GPU Acceleration**: Use CuPy for GPU-accelerated NumPy operations
4. **Mixed Precision**: Use float16 for faster computation

7.4 Evaluation Enhancements

1. **Per-Class Analysis**: Detailed breakdown of errors for each class
2. **Confidence Calibration**: Analyze prediction confidence distributions
3. **Failure Case Analysis**: Examine misclassified examples
4. **Grad-CAM Visualization**: Show which regions influenced decisions (if filters were trainable)

8 Bonus: Trainable Filters Implementation

To implement trainable convolutional filters, the following modifications would be needed:

8.1 Initialization

```
# Initialize 8 filters for Conv1, 16 for Conv2
self.conv1_filters = np.random.randn(8, 3, 3) * 0.1
self.conv2_filters = np.random.randn(16, 3, 3) * 0.1
```

8.2 Forward Pass

Apply all filters to create multiple feature maps:

$$F_k = \text{convolve2d}(X, K_k) \quad \text{for } k = 1, \ldots, n_{filters} \tag{37}$$

8.3 Backward Pass

Compute gradients with respect to filter weights:

$$\frac{\partial \mathcal{L}}{\partial K_{ij}} = \sum_{m,n} \frac{\partial \mathcal{L}}{\partial F_{mn}} \cdot X_{m+i,n+j} \tag{38}$$

This is equivalent to convolving the input with the upstream gradient.

8.4 Update Rule

$$K \leftarrow K - \eta \frac{\partial \mathcal{L}}{\partial K} \tag{39}$$

9 Conclusion

In this project, a neural network for pneumonia detection was implemented entirely from scratch using NumPy. Implementing convolution, pooling, and backpropagation manually required careful attention to tensor dimensions and gradient flow, but it helped reinforce the theoretical concepts covered in the course. The implementation included:

- Manual convolution and pooling operations
- Forward and backward propagation
- Weighted loss function for class imbalance
- SGD optimization with dropout regularization
- Comprehensive evaluation with multiple metrics

The model achieved reasonable performance on the test set, demonstrating that neural networks can be built and trained without relying on high-level deep learning frameworks. The exercise provided valuable insights into the inner workings of neural networks.

Key takeaways:

1. Hand-crafted features (Emboss/Sobel) provide interpretable but potentially suboptimal feature extraction
2. Class imbalance requires careful handling through weighted loss
3. Manual implementation provides deep understanding but sacrifices efficiency
4. There are many opportunities for improvement, particularly making filters trainable

Future work should focus on implementing trainable convolutional filters, adding more layers, and exploring advanced optimization techniques like Adam to improve performance and convergence speed.

Acknowledgments

This project was completed as part of CMPE 49T Fall 2025. The PneumoniaMNIST dataset is part of the MedMNIST collection, available at `https://github.com/MedMNIST/MedMNIST`.

A Code Structure

The implementation is organized into modular Python files:

- `main.py`: Main pipeline orchestration
- `neural_net.py`: Neural network implementation
- `data_utils.py`: Data loading and preprocessing
- `visualization.py`: Plotting and visualization functions

B Reproducibility

To reproduce the results:

1. Set random seed: `np.random.seed(42)`
2. Use the provided hyperparameters
3. Run: `python main.py --save-plots`

All plots and normalized data are saved for inspection.

C References

1. Yang, J., et al. (2023). "MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification." Scientific Data.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). "Deep Learning." MIT Press.

3. He, K., et al. (2015). "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification." ICCV.

4. Glorot, X., & Bengio, Y. (2010). "Understanding the difficulty of training deep feedforward neural networks." AISTATS.